

GCIS-124 Software Development and Problem Solving II

Instructor	Office	Phone	Email	Semester-Year
Danilo Kovacevic	D-106	2102	dxkcad1@rit.edu	Spring 2026
Office Hours:	Tuesday, Thursday: 12-1.30pm Wednesday: 11am- 12pm			

Discipline:	COMPUTING AND INFORMATION SCIENCES		Course Approval Date:				June 2021	
			Last Revision Date				July 2021	
Course Number:	GCIS -124	Course Title:	Software Development and Problem Solving II					
Undergraduate:	Yes	New Course:	Yes	Course Deletion:		X	Credits:	4
Graduate:	No	Prerequisites:	C- or better in SWEN-123 or CSEC-123 or ISTE-123 or equivalent course.		Co-requisites:	None		
Designated Innovation Journey Course:		Yes						

Recommended Books/Readings:

- 1- Provided Materials (lecture notes, assignments, code examples, etc.).
- 2- Recommended Text Books
 - Thomas, David, and Andrew Hunt. The Pragmatic Programmer: Your Journey to Mastery. Pearson Education, 2020, available online at RIT Libraries
 - Bloch, J., Effective Java, 2nd edition (3rd ed.). Addison-Wesley Professional, 2017, available online at RIT Libraries
 - Horstmann, Cay S. Big Java: Compatible with Java 5, 6 and 7" (4th edition) by Cay S. Horstmann, Wiley, 2010, available online at RIT Libraries
- 3- Online Resources
 - Java Web Site
 - <https://www.java.com/en/>
 - Java Online Documentation
 - <https://docs.oracle.com/en/java/javase/15/docs/api/index.html>
 - Project Euler
 - <https://projecteuler.net>
 - Git Documentation
 - <https://git-scm.com/doc>
 - Code Academy Git Course
 - <https://www.codecademy.com/learn/learn-git>

Official Course Description:

A second course that delves further into computational problem solving, now with a focus on an object-oriented perspective. There is a continued emphasis on basic software design, testing & verification, and incremental development. Key topics include theoretical abstractions such as classes, objects, encapsulation, inheritance, interfaces, polymorphism, software design comprising multiple classes with UML, data structures (e.g. lists, trees, sets, maps, and graphs), exception/error handling, I/O including files and networking, concurrency, and graphical user interfaces. Additional topics include basic software design principles (coupling, cohesion, information expert, open-closed principle, etc.), test driven development, design patterns, data integrity, and data security.

The following concepts will be covered primarily during lectures:

- 1- Algorithmic thinking, computational problem solving
- 2- Software design (UML), design principles and design patterns
- 3- Test Driven Development (TDD), unit & automated testing, command line usage, team work
- 4- Distributed version control (working in teams, e.g. branching, merging, pull requests)
- 5- Classes, objects, constructors, fields, methods
- 6- Encapsulation, access modifiers, packages
- 7- Inheritance, interfaces, generics, abstract classes, lambdas
- 8- Multi-dimensional arrays, lists, queues, binary trees, maps, sets, graphs

The following topics are introduced and reinforced in every assignment through practice.

- 1- A systematic approach to computational problem solving using software engineering including informal and formal approaches to problem solving
- 2- Algorithmic thinking, computational problem solving

Course Learning Outcomes:

At the end of this course, a student will be able to:

Describe the basic abstractions and constructs of a selected contemporary, object-oriented programming language.
Describe and apply problem-solving skills that are appropriate to solve a variety of computing problems of varying degrees of difficulty and complexity.
Describe and apply a variety of algorithms and data structures to implement solutions to a variety of computing problems of varying degrees of difficulty and complexity.
Apply fundamentals of software engineering including software design, testing, and incremental development.

Mapping of CLOs to PLOs: Please refer to the link below to see the mapping of the Course Learning Outcomes to the Program Learning Outcomes:

CIT-

https://docs.google.com/spreadsheets/d/1M-utTLZAyBlqg2OtdUHHBfmSOTbK4_33/edit?usp=sharing&ouid=101887271551547812808&rtpof=true&sd=true

CSEC-

https://docs.google.com/spreadsheets/d/1atWAsEkcFRVHDW_vUIXKSVj2AxRJ7GqV/edit?usp=share_link&ouid=111309766900233126438&rtpof=true&sd=true

Innovation Outcomes:

Innovation Outcome	Associated CLO	Assessment Method
CLO1: ARTICULATE the difference between design thinking, innovation and entrepreneurship and possess a shared vocabulary for the process elements of each term.	CLO4: Apply fundamentals of software engineering including software design, testing, and incremental development.	Assignment #3

Teaching & Learning Methodologies:

This course aims for you to learn to program in a selected, contemporary, high-level programming language (e.g. Java). You will describe and apply problem solving skills, algorithms, and data structures that are appropriate to solve a variety of computing problems of varying degrees of complexity. You will describe and apply fundamental concepts of software engineering including understanding needs, software design, solution testing, and incremental development.

This course will combine lecture and in-class learning activities, problem solving in teams of 3-4 students, and out of class assignments.

This course is the second in a year-long introductory sequence.

This course meets for six hours each week.

Group Work

When assigned to a group for some of the in-class activities, each student in the group is expected to contribute to the required activities. Peer evaluation of group members will constitute a portion of the overall participation grade for the course. Group members who DO NOT contribute adequately to the assignment will receive NO credit for the assignment.

Modifications to Course Materials

Any or all of the previous information is subject to change or modification during the semester.

Tips for Success

- Spend ~30 minutes reviewing the lecture materials before starting the homework assignments. Practice the coding activities to see if you can solve them without looking at the answers. If you cannot solve one or more problems, ask for help!
- Read the assignment fully before asking for help! Make a note of instructions that raise questions for you. Visit your instructor during their office hours.
- Do not try to solve a large problem all at once. Divide the problem into smaller parts and solve each one individually, building a solution incrementally.
- Refer to your team's problem-solving solution!
- If you're not sure how to do something, look for a similar example in the class activities.
- Understand that you have the tools that you need to solve the problem; the skills you are developing will help you recognize which tools can be used to solve which problems.

Grading Scheme

Excellent	Good	Satisfactory	Minimal Pass	Fail
	B+ 87-89.99	C+ 77-79.99	D 60-69.99	F 0-59.99
A 94-100	B 83-86.99	C 73-76.99		
A- 90-93.99	B- 80-82.99	C- 70-72.99		

Coursework and Distribution:

The final grade for the course will be calculated based on the following assessments:

Assessment Instrument	CLO Mapping	Weight
Quizzes 4 (5% each)	CLO 1	20%
Assignments 3 (10%, 10%, 10%)	CLO 4,3,2	30%
Midterm	CLO 2,3,4	20%
Final Exam	CLO 1,2,3,4	30%
Total		100%

Description of Assessments

Lecture

1. Lecture - The presentation of new material regarding that day's topics. While the format of the presentation is up to the individual instructor, a standard deck of slides will be provided. During the lecture, activities will also be conducted.
2. Problem Solving - After the lecture, students will be divided into groups of 3-4 by their instructor and given a series of problems to solve together.

Practica

This course has 2 exams: one midterm practical, and a final exam.

Midterm practical will be a timed test of design, implementation, and testing skills administered during on class period. The final practical will be cumulative and will be administered during the scheduled final exam period. There may also be a written component to the exam. The entire exam will be designed to be completed in under 2 hours, but students will be given the entire final exam period to complete it.

Assignments

During the semester students will be given assignments to be completed before the deadline tbd. In many cases, they will be original assignments that use the same concepts or principles. Students will have a minimum of 2 days to complete each assignment (e.g. Monday to Wednesday). Submission must be done only through the MyCourses. Late submissions may be considered in *48 hours and in such cases 25% of the total grade will be deducted*.

The last 20-30 minutes of class on most days will be devoted to team problem solving, during which students are placed onto teams of 3-4 by their instructor and presented with a series of problems to solve together. Every student on the team is expected to contribute to the team's solution.

Problem solving is graded based on contributions to the final solution by each team member. Students should contribute to their team's solution by actively engaging in the discussions. Provided that a student is present and contributing to their team's solution, they will receive full credit of the final team activity grade.

Assignments will be graded according to the rubric which will be explained in the lecture. There might be a presentation related to the assignment.

Course Calendar:

Week/ Day	Topic	Assessments
1	Python to Java	
2	Java Basics and Classes	
3	Inheritance	Quiz 1
4	GUI	Assignment 1
5	Data Structure I	
6	Data Structure II	Assignment 2
7	Data Structure III	Quiz 2
8	Revision Week	Midterm Exam
9	Algorithms	
10	Lambdas	Quiz 3
11	Concurrent Programming	Assignment 3
12	Thread Cooperation	Quiz 4
13	Network Programming	
14	Revision	
15	Final Exam	

Course Policies:*Office Hours:*

While it's OK to visit your instructor just to chat, if you are looking for help with your coursework, it is always best to come prepared.

- At a minimum, you should prepare a list of questions that you would like to ask; consider writing or typing these questions in advance.
- If you would like help solving a problem in a program that you have written, make sure that your laptop is charged, and that your program is open and ready to review.
- Be prepared to answer the following questions:
 - What did you expect to happen?
 - What actually happened?
 - What have you tried to solve the problem yourself?

Attendance Policy:

Attendance is mandatory for this class. Students with regular absences will most likely be unsuccessful in this course. Therefore, students who accumulate more than a certain number of session absences (more than 30%) will be automatically assigned an "F" grade regardless of their assessment results. This is applied per university policy 3.1.14-B.

Definition of a session attendance: It is being present from the very start of the class session until the very end. Arriving to class late or leaving early will be counted as an absence. **Attendance** is essential to understanding and progress. You will miss important material if you are absent, and I will not re-teach it to you privately. If you miss a class or are going to be absent, let me know and arrange to meet with peers to discuss what you missed.

Policy on the Use of AI in this Course

In this course, we aim to equip you with fundamental skills and foster critical thinking, creativity, and independent problem-solving. Therefore, the use of AI software tools to generate all or part of a solution for any assessment or exam is **strictly prohibited**, *unless explicitly stated otherwise by the instructor for specific assignments or activities*.

Permitted AI Use:

There may be instances where the instructor designates specific assignments or activities for which AI tools are permitted. In such cases, the instructor will provide clear guidelines on the acceptable use of AI. **Any use of AI outside of these clearly designated areas will be considered a violation of the Academic Integrity Policy.**

Penalties for Unauthorized AI Use:

- Any unauthorized use of AI will result in reporting through the University Student Academic Integrity Violation (SAIV) System, and the designated policy penalty will be applied accordingly.

Why This Policy?

This policy is designed to ensure you develop a strong foundation in the course material and cultivate essential skills independently. While AI tools can be powerful, over-reliance on them, particularly in the early stages of learning, can hinder your ability to grasp fundamental concepts and develop critical thinking abilities. Practice is paramount to mastering these skills. Our goal is to help you become adept critical thinkers and creators, not simply experts in using AI tools. Students who depend on AI for homework or exercises often find themselves unprepared for exams and subsequent courses.

For more details on academic dishonesty, please refer to the relevant section later in this Syllabus.

Academic Honesty:

[RIT's Academic Integrity Policy](#) defines the basic forms of academic dishonesty (cheating, duplicate submission, and plagiarism) and explains the official RIT policy regarding academic dishonesty.

Apart from the problem solving activity during lecture, all work in this course is individual. All other quizzes, assignments, and practica are expected to be the result of individual effort, not teamwork or collaboration. Students are permitted to discuss assignments with other students provided that it is not at a level of detail that results in more than one student writing identical solutions. In other words, high level discussion of topics, approaches, or algorithms with other students are acceptable, but low level implementation details (such as line-by-line descriptions of a solution) are prohibited. For Problem Solving activities, in-group discussions allowed but discussion with other group is strictly prohibited.

Submitting work that was authored all-or-in-part by another individual is considered a violation of academic honesty. Conversely, providing all-or-part of a solution to another student is also a violation of academic honesty. If a student is suspected of cheating, the instructor shall notify the students involved and conduct an investigation in accordance with RIT's Academic Integrity Policy.

If the investigation conducted by the instructor determines that the student violated RIT's Academic Integrity Policy, the minimum penalty is a 0 for an assessment on which the student cheated. Multiple and/or egregious violations will result in a failing grade in the course and a note in the student's permanent record. Subsequent offenses (e.g. in later courses) will be treated as additional instances of academic honesty and may result in a failing grade in the course (even for a single offense) or expulsion from the program or university.

We assume that works submitted by students – low-stake writings, drafts, final versions, and all other submissions – are generated by students themselves. Therefore, the use of AI Writing tools is prohibited at all stages of this class's assignments; utilizing such tools would constitute academic dishonesty and a breach of RIT's Academic Integrity Policy.

GCIS-123

Refer to Policy 3.1.7- Policy on Academic Responsibility (section 2. Academic conduct) in the Student Handbook.

RIT Dubai Honor Principles:

Refer to Policy 5.1.1- RIT Dubai Honor Code in the Student Handbook

Academic Integrity:

Refer to Policy 3.1.7- Policy on Academic Responsibility (section 2. Academic conduct) in the Student Handbook.

Procedures for handling Academic Dishonesty:

Refer to Section 2. Academic Conduct of Policy 3.1.7- Policy on Academic Responsibility in the student handbook.